

문서함에서 위키까지

각자 자기 private repo에 자기 vault를 만든다. 오늘은 그 vault에 문서 한 건이 들어가 위키 노트가 되는 경로를 끝까지 본다. 개념 20분은 그 경로가 왜 그렇게 생겼는지를 다룬다.

20'

개념 — 컨텍스트 · Skill · MCP

40'

시연 — 변환 · 위키화 · 활용

1건

과제 — 내 문서로 직접

목차

01	오늘의 목표	3'	02	컨텍스트	9'	03	Skill	5'
04	MCP	3'	05	사전 요구사항	3'	06	한글 문서 변환	11'
07	다른 포맷의 경로	5'	08	위키로 쌓기	7'	09	업무에 쓰기	9'
10	과제	3'	11	다음 주	2'			

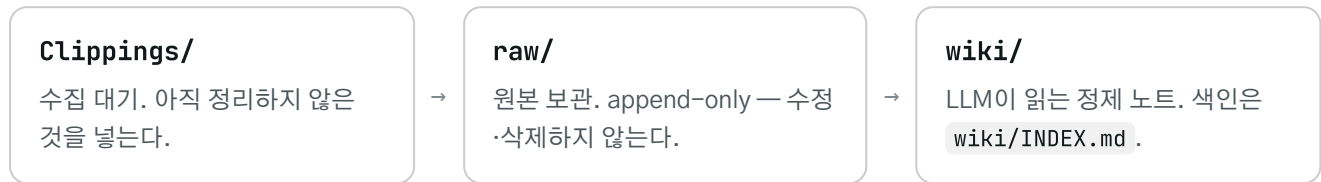
개념 20' · 시연 40'

합계 60' · 20블록

이 섹션을 보면 자기 vault가 지금 어떤 상태인지, 오늘 끝나면 무엇이 늘어나는지 안다.

문서함에서 위키까지 3계층

vault는 세 칸으로 나뉜다. 칸마다 규칙이 다르다.



1주차에 각자 만든 vault는 지금 이 상태다.

항목	값
wiki/ 아티클 / 토픽	4개 / 2개
wiki/INDEX.md	915 B
wiki/VAULT_MEMORY.md	2,781 B
raw/	1건
Clippings/	0건

wiki 노트 4개는 1주차에 클리핑 1건을 넣은 결과다. 오늘 끝나면 여기에 자기 업무 문서가 raw/ 와 wiki/ 양쪽으로 들어간다.

이 섹션을 보면 자기 vault 문서를 왜 짧게 유지해야 하는지, 어느 파일을 먼저 줄여야 하는지 숫자로 판단할 수 있다. 뒤에 나오는 Skill과 MCP도 여기서 나온 예산 개념 위에서 있다.

모델의 작업 기억

컨텍스트는 모델이 답을 만들 때 참조하는 텍스트 전체다. Anthropic 공식 문서는 이것을 학습 데이터와 구분해 "작업 기억"이라 부른다.

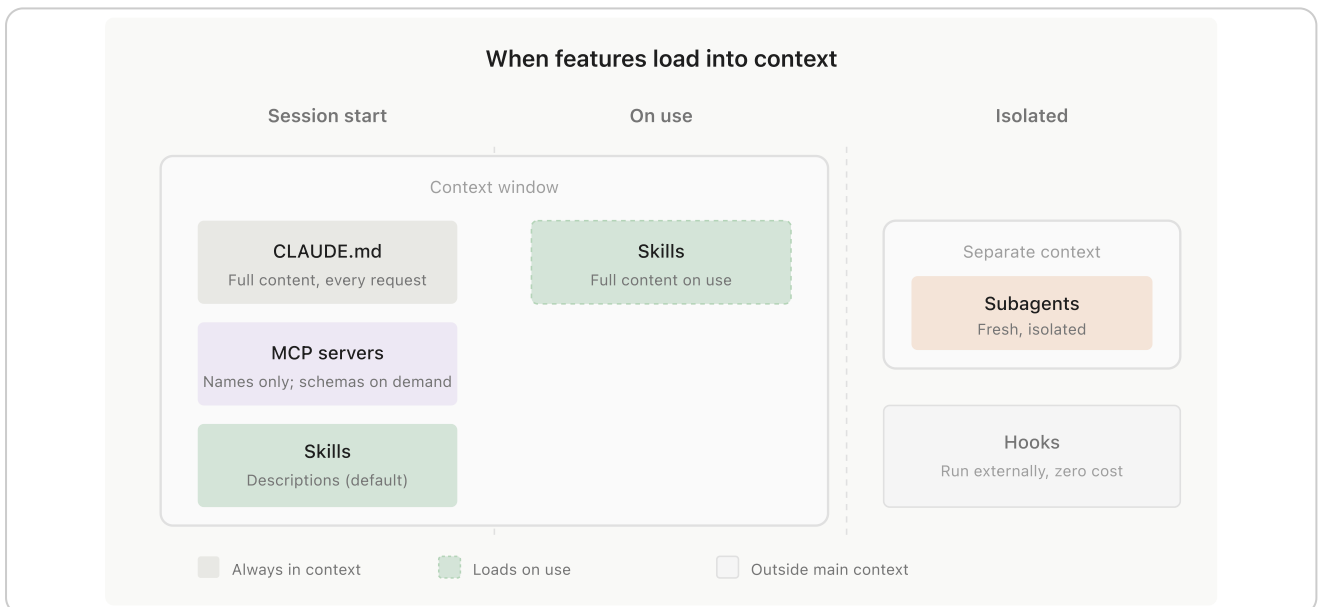
"The 'context window' refers to all the text a language model can reference when generating a response, including the response itself. ... instead represents a 'working memory' for the model."

학습 데이터는 이미 모델 안에 있는 것이고, 컨텍스트는 지금 책상에 펼쳐 놓은 것이다. 자기 vault 문서는 학습 데이터가 아니다. 매번 책상에 다시 올린다.

책상에 올라가는 것은 자기가 타이핑한 글자만이 아니다.

"Everything in the request counts toward the context window: the system prompt, every message in **messages** (including tool results, images, and documents), and your tool definitions."

시스템 프롬프트, 도구 정의, 읽어들이는 파일이 전부 같은 자리를 쓴다. 그래서 규칙 파일을 짧게 쓰라는 말은 취향이 아니다. 자리 문제다.



인용 When features load into context — 무엇이 항상 실리고, 무엇이 쓸 때만 실리고, 무엇이 밖에 있는지. 오늘 다루는 세 개념이 이 그림의 세 칸에 각각 들어간다. 출처 Anthropic Claude Code 문서 (code.claude.com). 사내 교육 인용.

우리 vault의 고정 비용 21,293 B

우리 vault는 대화 첫 글자를 치기 전에 이미 21,293 B를 쓴다. `wc -c` 로 직접 잴 값이다.

구분	파일	BYTES	누구나 같은가
자동 로드	~/ .claude/CLAUDE.md	3,298	개인차
자동 로드	~/ .claude/rules/lemon-rules.md	1,460	개인차
자동 로드	CLAUDE.md	2,128	vault 공통
자동 로드	MEMORY.md	496	개인차
자동 로드 소계		7,382	
지시로 읽음	VAULT_RULES.md	10,215	vault 공통
지시로 읽음	wiki/VAULT_MEMORY.md	2,781	vault 공통
지시로 읽음	wiki/INDEX.md	915	vault 공통
지시로 읽음 소계		13,911	
합계		21,293	

이 표는 발표자 환경 실측이다

오른쪽 칸을 함께 봐야 한다. vault가 가져오는 몫은 **16,039 B**로 누구나 같고, 나머지 **5,254 B**는 각자의 개인 설정 (~/ .claude/)이라 수강자 머신마다 다르다. 자기 값은 /context 로 직접 쟀다. 합계 숫자를 자기 값으로 옮겨 적지 않는다.

두 소계를 나눈 것이 핵심이다. 성격이 다르다.

- **자동 로드** — 세션 시작 시 무조건 실행된다. 대화가 길어져 compaction이 일어나도 디스크에서 다시 들어온다.
- **지시로 읽음** — CLAUDE.md 의 Before vault work, read: 목록이다. @import 가 아니라 "읽어라"는 지시이므로 vault 작업이 시작될 때 Read로 들어온다. **compaction 후에는 요약되어 사라진다.**

VAULT_RULES.md 하나가 10,215 B다. 자동 로드 소계 전체보다 크다. 그래서 자동 로드에서 빼고 필요할 때 읽게 만들었다. 대신 절대 잊혀선 안 되는 규칙 세 개(raw/ append-only, 개인 실험 데이터 커밋 금지, 8 KB 상한)는 CLAUDE.md 의 ## Hard Invariants 에 중복으로 적어 뒀다. compaction 후에도 남는 쪽에 사본을 둔 것이다.

숫자로 박아 둔 예산

예산은 감각이 아니라 숫자다. 우리 vault는 세 군데에 숫자를 박아 뒀다.

1. **상한 하나.** VAULT_RULES.md 가 wiki/VAULT_MEMORY.md 에 8 KB(8,192 B) 상한을 걸었다. 현재 2,781 B로 상한의 33.9%다. wc -c 바이트 기준이라 lint가 기계적으로 검사한다. 매 실행 서술을 memory에 붙이면 금방 찬다. 그래서 실행 이력은 docs/vault-ingest-log.md 로 분리했다.
2. **색인은 작게, 본문은 나중에.** 같은 형태가 vault에 세 번 반복된다.

대상	상시 로드	전량	비율
스킬 17개 (메타데이터만)	6,823 B	188,371 B	3.6%
auto memory (MEMORY.md 만)	496 B	5,603 B	8.9%
wiki (INDEX.md 만)	915 B	노트 전량	색인 915 B

3. 원본과 정제본의 분리. 여기서 raw/ 와 wiki/ 를 나눈 이유가 나온다. raw/ 는 참고이고 wiki/ 는 책상이다. 원본을 그대로 책상에 올리면 자리가 모자란다. 그래서 원본은 raw/ 에 그대로 두고, 컨텍스트에 올릴 정제본만 wiki/ 에 따로 만든다. 이 분리가 오늘 시연할 변환 → 인제스트 경로의 이유다.

토큰 수를 쓰지 않는 이유

한국어가 영어보다 토큰을 더 쓰는 방향은 확인되지만, 배수의 공식 수치는 없다. 필요하다면 /context 로 자기 세션을 직접 재서 자기 수치를 만든다.

이 섹션을 보면 스킬이 어떤 파일이고 언제 켜지는지 안다. 뒤에 시연할 변환도 스킬 하나가 켜지면서 시작한다.

파일 하나로 된 스킬

스킬은 마크다운 파일 하나다. 구조는 두 부분이다 — YAML 프론트매터(`name` · `description`)와 본문. 필수 필드는 `name` 과 `description` 둘뿐이고, 나머지는 각자의 관례다.



인용 A simple SKILL.md file — 위 칸이 프론트매터, 아래 칸이 본문. 출처 Anthropic Agent Skills 문서 (platform.claude.com). 사내 교육 인용.

우리 vault의 스킬도 같은 형태다.

```
# config/skills/hwp2md-ingest/SKILL.md
---
name: hwp2md-ingest
description: HWP/HWPX 문서를 vault 인제스트 가능한
  마크다운으로 변환한다. ...
---
```

여기서 `description` 이 발동 조건이다. Claude는 사용자 요청을 이 문장과 대조해 스킬을 켜지 정한다. 공식 문서도 그렇게 적는다.

“The `description` is what Claude matches your request against when determining whether to trigger the Skill, so it must say both what the Skill does and when to use it.”

비유가 아니다 — 우리 VAULT에 안 걸리는 스킬이 하나 있다

`config/skills/ollama-local-models.md` 에는 프론트매터가 아예 없다. 첫 줄이 `# Skill: ollama-local-models (v1.0.0)` 이고 `name` · `description` 문자열이 파일 어디에도 없다. 1단 메타데이터가 0이므로 Claude가 이 스킬의 존재를 알 방법이 없다. 본문이 3,035 B로 잘 쓰여 있어도 켜지지 않는다.

스킬을 만드는 법은 4주차에 다룬다. 오늘은 읽는 법까지다.

3단 로딩과 상시 비용 3.6%

스킬을 많이 두면 무거워진다는 말은 사실이 아니다. 켜지기 전에 컨텍스트를 차지하는 것은 `name` 과 `description` 뿐이다.

Level	File	Context Window	# Tokens
1	SKILL.md Metadata (YAML)	Always loaded	~100
2	SKILL.md Body (Markdown)	Loaded when Skill triggers	<5k
3+	Bundled files (text files, scripts, data)	Loaded as-needed by Claude	unlimited*

인용 Progressive disclosure levels — 1단 약 100토큰, 2단 5천 토큰 미만, 3단 필요할 때만. 출처 Anthropic 엔지니어링 블로그 “Equipping agents for the real world with Agent Skills” (2025-10-16). 사내 교육 인용.

우리 스킬 17개로 재면 이렇게 나온다.

단계	무엇이 로드되나	크기	전량 대비
1단 (상시)	17개의 <code>name</code> · <code>description</code> 값	6,823 B	3.6%
2단 (발동 시)	해당 스킬의 진입점 <code>.md</code> 1개	2,678 ~ 18,011 B	—
3단 (필요 시)	스킬 폴더의 참조 문서·스크립트	접근 전 0	—
전량	스킬 폴더 전체	188,371 B	100%

샘범을 밝힌다

3.6%는 `name` · `description` 값 텍스트만 센 값이다. --- 구분선을 포함한 프론트매터 블록 전체를 세면 8,508 B = 4.5%가 된다. 세션 시작에 실제로 주입되는 것은 값 텍스트이므로 3.6%를 쓴다.

3단 구조가 눈에 보이는 예가 `pdf2md-ingest` 다. 진입점 `SKILL.md` 는 8,600 B인데 폴더 전체는 42,348 B다. PDF 변환을 시킬 때 들어오는 것은 8,600 B뿐이고, `scripts/` 3개와 `design.md` · `implementation-plan.md` 는 필요할 때만 읽힌다. 설계 문서를 폴더에 넣어도 평소 비용이 없다.

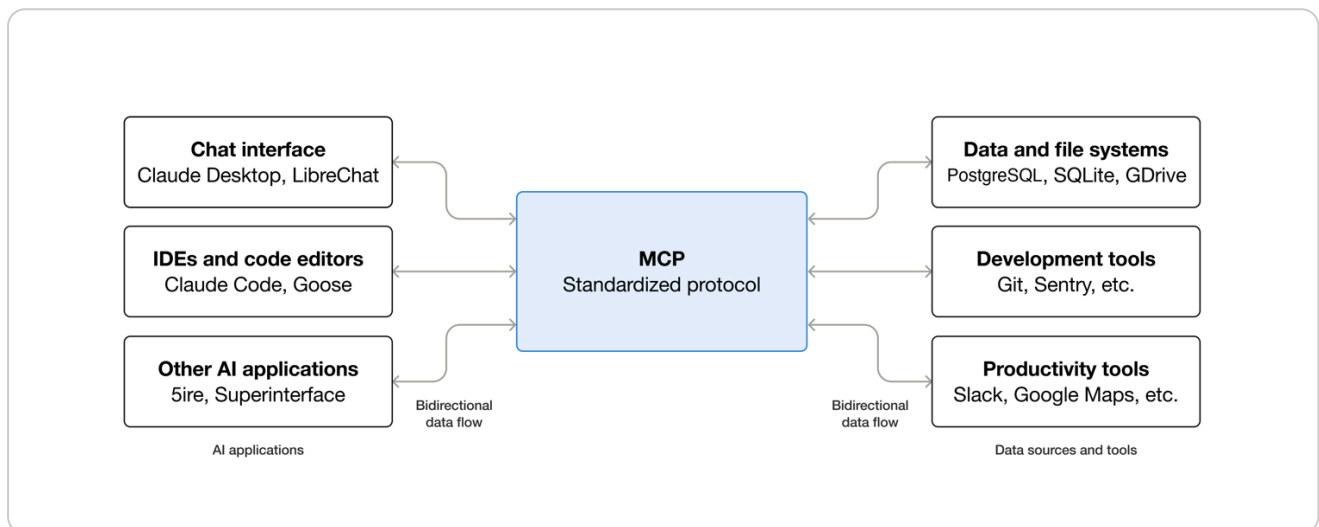
오늘 시연할 `hwp2md-ingest` 도 같은 구조다 — `SKILL.md` 8,991 B, 폴더 전체 10,832 B. `.hwp` 변환을 요청하면 이 8,991 B가 그때 들어오고, 그 안의 전략 분기표대로 변환 경로가 정해진다. 6번 시연에서 이 동작을 화면으로 본다.

이 섹션을 보면 MCP가 무엇을 표준화한 것인지, 무엇이 MCP가 아닌지 구분할 수 있다.

AI 앱의 USB-C 포트

MCP(Model Context Protocol)는 AI 애플리케이션을 외부 시스템에 붙이는 오픈소스 표준이다. 공식 문서가 직접 USB-C에 비유한다.

"Think of MCP like a USB-C port for AI applications. Just as USB-C provides a standardized way to connect electronic devices, MCP provides a standardized way to connect AI applications to external systems."



CC BY 4.0 MCP 구조 — 왼쪽 AI 앱, 오른쪽 데이터·도구, 가운데 하나의 규격. 출처 modelcontextprotocol.io. 저작자 표시 조건.

규격이 하나라 붙이는 방법이 하나다. 우리 vault에서는 `config/skills/google-workspace.md`가 그 실물이다.

```
uvx workspace-mcp --tools drive sheets slides
```

이 한 줄로 Drive·Sheets·Slides가 붙는다. 세 서비스를 각각 커스텀 스크립트로 만들지 않았다. 서버 하나가 세 서비스의 도구를 한꺼번에 들여온다.

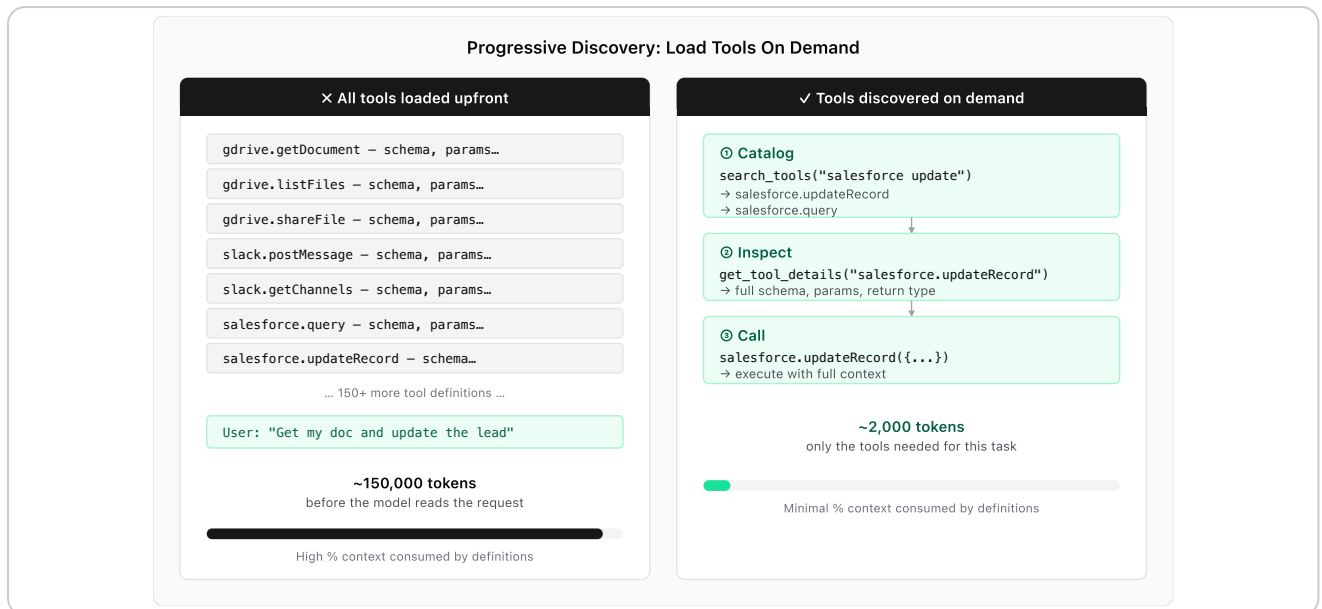
붙었다고 다 MCP는 아니다

같은 폴더의 `ollama-local-models.md`는 로컬 모델을 쓰지만 MCP가 아니다. 파일에 `grep -ci mcp`를 걸면 0이 나온다. 대신 HTTP API를 직접 호출한다.

```
curl http://127.0.0.1:11434/api/generate
```

외부 시스템을 붙이는 방법은 여럿이고, MCP는 그중 규격이 정해진 하나다. 이 구분을 못 하면 "연결됐다"는 말과 "MCP로 연결됐다"는 말을 섞어 쓴다.

도구를 붙일 때도 컨텍스트 예산이 걸린다. 도구 정의를 전부 미리 올리면 모델이 요청을 읽기도 전에 자리를 다 쓴다.



CC BY 4.0 도구를 다 올리면 — 도구 정의 150개가 약 150,000 토큰. 모델이 요청을 읽기 전에 이미 쓴다. 그래서 이름만 싣고 스키마는 요청할 때 가져온다. 출처 modelcontextprotocol.io § Client Best Practices.

3주차가 이 클라우드 연동이다 — 드라이브·슬라이드·시트·메일, 텔레그램/카카오톡. 오늘 MCP는 그 준비다.

내 문서의 확장자에 따라 필요한 도구가 다르다. 이 슬라이드에서 수강자는 자기 문서가 오늘 바로 되는 경로인지, 설치가 먼저 필요한 경로인지 판단한다.

내 문서에 필요한 도구

한글 문서가 가장 쉽다. `uv` 하나만 있으면 되고 한컴오피스는 필요 없다. HWPX가 국가표준 KS X 6101 (OWPML)을 따르는 ZIP+XML 포맷이라 순수 Python으로 읽힌다.

내 문서	필요 도구	MACOS 설치	이 머신 상태
<code>.hwp</code> <code>.hwpX</code>	<code>uv</code>	<code>brew install uv</code>	설치 없이 동작 — 실증됨
<code>.pdf</code>	<code>uv + poppler</code>	<code>brew install poppler</code>	있음 — 스캔 PDF까지 실증됨
<code>.docx</code>	<code>pandoc</code>	<code>brew install pandoc</code>	미설치 — 오늘 라이브 불가
<code>.doc</code> (구형)	LibreOffice 또는 Word	<code>brew install --cask libreoffice</code>	Windows 실행기 미검증

Windows는 아래 두 줄을 쓴다. 설치가 끝나면 새 터미널을 연다. 기존 터미널에는 경로가 잡히지 않는다.

```
winget install astral-sh.uv
winget install JohnMacFarlane.Pandoc
```

변환 스킴 3종은 전제 도구가 없으면 안내만 하고 중단한다. 자동으로 설치하지 않는다. 그래서 도구가 없으면 문서가 망가지는 대신 아무 일도 일어나지 않는다.

.hwp 샘플 1건을 채팅 요청부터 검수까지 시연한다. 게이트 3개를 먼저 통과하고, 판정 수치로 변환 결과를 확인하고, 마지막에 눈으로 검수한다. 이 순서가 그대로 과제 절차다.

요청 문구와 게이트 3개

수강자는 명령을 외우지 않는다. 채팅에 아래 두 줄을 적으면 스킵이 걸린다. 두 번째 줄은 파일 경로다.

이 한글 파일 마크다운으로 변환해서 Clippings에 넣어줘
projects/second-brain/samples/점검-결과-보고.hwp

변환이 시작되기 전에 게이트 3개가 순서대로 걸린다. 하나라도 실패하면 아무것도 쓰지 않고 사유만 보고한다.

- 01 **커밋 가능성** — 화면에는 이 문구가 뜬다: "이 문서는 팀 공유 vault에 커밋 가능한가?" 고객사·개인 문서면 정식 인제스트를 중단하고 vault 밖 변환 모드를 제안한다. 수강자가 그 모드를 명시적으로 고른 경우에만 변환한다.
- 02 **VAULT_DIR resolve** — 사용자 명시값이 1순위, vault 구조가 확인된 현재 루트가 2순위, 그 외에는 질문한다. ~/knowledge 로 조용히 넘어가지 않는다.
- 03 **중복** — raw/hwp/<원본파일명> 이 이미 있으면 중단하고 보고한다. raw/ 는 append-only다.

화면 문구를 읽는 법

오늘 교육은 각자 개인 private repo에 자기 vault를 만드는 전제다. 화면에는 "팀 공유 vault"로 뜨므로, 그 문구를 "git 이력에 영구히 남겨도 되는가"로 읽는다.

게이트 1에서 막히는 문서에는 vault 밖 변환 모드가 있다. 중복 검사를 생략하고, 추출과 판정은 똑같이 하되 **vault 아래에 아무것도 쓰지 않는다**. 검증 기준도 "산출 경로가 \$VAULT_DIR 밖인가"로 뒤집힌다. 이 모드의 산출물에서 나온 결과를 나중에 vault로 넣을 때는, 커밋 전에 연락처·이메일·사업자번호·상세 주소가 섞이지 않았는지 diff로 확인한다. 완료 보고에는 모드 명칭과 산출 경로가 함께 남는다.

한 문장으로

"회사 문서를 넣어도 되나"의 답은 정책 문장이 아니라 경로로 준비돼 있다.

변환 판정 수치

게이트를 통과하면 추출이 돌아간다. 화면에 남는 로그는 이 4줄이다.

```
$ uv run scripts/h1-extract.py 점검-결과-보고.hwp out.md
Installed 5 packages in 9ms
wrote 557 chars → out.md
stats chars=403 tables=1 images=0 encrypted=False
실행 시간 0.465s
```

사전 설치는 0건이다. `uv` 가 의존성 5개를 9ms에 준비하고, 변환 자체는 0.465초에 끝났다.

세 번째 줄과 네 번째 줄의 숫자가 다른 이유는 세는 방식이 다르기 때문이다. `wrote 557` 은 마크다운 전체 길이이고, `stats chars=403` 은 공백을 뺀 본문 문자 수다(`len("".join(md.split()))`). 판정에 쓰는 값은 뒤쪽이다.

마지막 `stats` 줄이 판정 근거다. 채택 임계는 `chars ≥ 300` 이고 이 문서는 403자라 H1 경로가 채택됐다. `chars=403 tables=1` 은 `samples/README.md` 에 적힌 기대값과 정확히 일치한다.

표는 마크다운 표로 재구성됐다.

설비코드	점검 항목	측정값	판정	
CMP-02	토출 압력	7.1 bar	적합	
CMP-03	진동 속도	5.2 mm/s	부적합	

수치 두 개만 보면 된다

`tables=0` 이면 표가 문단으로 풀렸다는 뜻이다. `headings=0` 이면 제목 구조를 잃은 것이다. 둘 중 하나라도 0이면 위키로 넘기지 않는다.

머리말 중복 검수

같은 실행에서 결함이 나왔다. 산출 마크다운의 첫 줄이 이렇다.

```
제3공장 설비팀 - 대외비 아님 (교육용 샘플)압축기 정기 점검
결과 보고제3공장 설비팀 - 대외비 아님 (교육용 샘플)
```

머리말이 본문 제목과 붙어 두 번 들어갔다. `chars=403 tables=1` 은 정상으로 나왔는데도 이렇다. 한글 문서의 머리말·꼬리말은 본문과 다른 층에 있어서, 텍스트로 뽑을 때 본문에 섞인다.

오늘 가장 중요한 한 줄

변환은 자동이지만 검수는 자동이 아니다. 위키로 넘기기 전에 첫 줄과 표를 눈으로 본다. 이 2가지만 보면 된다.

이 결함이 그대로 위키에 들어가면 검색이 오염된다. 그래도 되돌릴 수 있다. 원본이 `raw/` 에 그대로 남아 있어서, 잘못 만든 노트를 버리고 다시 만들면 된다.

내 문서가 한글 문서가 아닐 때 어디로 가는지 정리한다. 이 슬라이드에서 수강자는 자기 문서의 경로와, 그 경로가 오늘 실증된 것인지 아닌지를 확인한다.

포맷별 변환 경로

문제는 한글 파일이 아니다. 바이너리라는 사실이다. `.doc` · `.pdf` · `.xls` 도 같은 문제를 갖는다. 그래서 앞단 경로가 포맷별로 갈라진다.

내 문서	경로	오늘 상태
<code>.hwp</code> <code>.hwpk</code>	H1 — 순수 Python 추출	실증됨 (06에서 시연)
텍스트 PDF	S2 — 텍스트 추출	실증됨
스캔 PDF	S6 — 로컬 OCR (또는 S4 비전 전사)	실증됨 (2026-08-28)
<code>.docx</code>	D1 — pandoc 직행	미실증 — <code>pandoc</code> 미설치
<code>.doc</code> (구형)	D2a·D2b·D2c 세 갈래	Windows 실행기 미검증

PDF는 **페이지당 300자**로 갈린다. 샘플 2개가 기준선 양쪽에 있다. `안전-교육-자료.pdf` 는 499자/페이지라 추출로 가고, `점검-기록지-스캔.pdf` 는 0자라 OCR로 간다.

스캔 PDF는 로컬 OCR로 실증됐다. `pdftoppm` 이 페이지를 150dpi로 렌더하고, `ppu-paddle-ocr` 의 한글 모델(v5-korean-mobile)이 표까지 읽어냈다.

조용한 빈 파일은 결함이다

스캔 PDF에서 아무 말 없이 빈 파일이 나오면 그건 결함이다. 스킵은 0자를 감지하면 멈추고 다른 경로를 제안한다.

`.docx` 는 pandoc이 표·헤딩·목차를 보존한다. 샘플 `설비-점검-절차.docx` 의 기대값은 헤딩 6 · 표 1(20셀) · 불릿 3 · 이미지 1 · 316자다. **이 값은 2026-08-25 기록값이고 오늘 이 자리에서 실증한 값이 아니다.** `pandoc` 이 없어 라이브로 못 돌린다.

포맷을 바꾸는 효과는 정확도만이 아니다. 같은 표를 어떤 포맷으로 직렬화하냐에 따라 토큰이 최대 3배까지 차이 난다.

이 섹션을 보면 변환이 끝난 파일이 어떤 판단을 거쳐 위키 노트가 되는지 안다. 인제스트가 모든 파일을 노트로 만들지 않는다는 것이 요점이다.

변환과 위키화의 분리

변환 스킬은 `Clippings/`에 마크다운을 놓는 데까지만 한다. 위키화는 `vault-ingest`가 이어받는다. 한 스킬이 둘 다 하지 않는 이유는 판단의 성격이 다르기 때문이다. 변환은 포맷을 바꾸는 일이고, 위키화는 무엇을 남길지 고르는 일이다.

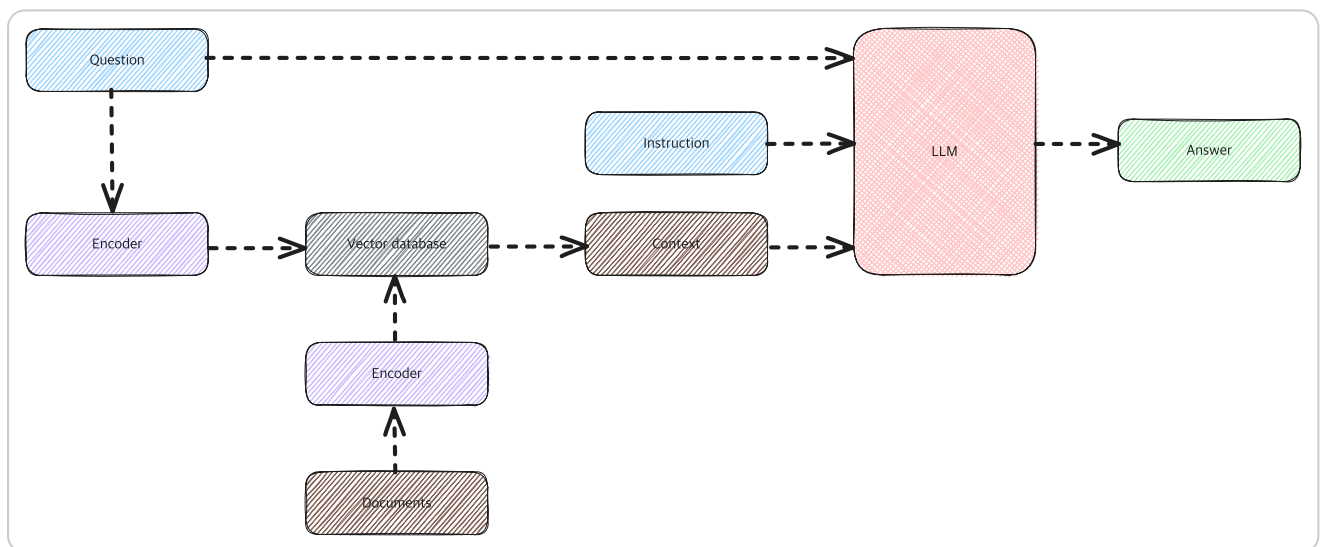
파일은 세 자리를 지난다.

- `Clippings/` — 변환된 마크다운이 들어온다. 위키화를 기다리는 자리다. 처리되면 파일이 `raw/`로 옮겨져 비워진다
- `raw/` — 원문을 그대로 보관한다. 이후 수정하지 않는다
- `wiki/` + `wiki/INDEX.md` — 다시 꺼내 쓸 개념만 노트로 쓰고 색인에 한 줄 올린다

인제스트 트리거는 채팅에 `인제스트해줘` 한 줄이다.

노트에는 출처가 `sources: ["raw/<파일명>.md"]` 문자열로 박힌다. 나중에 "이건 어디서 나온 얘기지"를 원문까지 되짚을 수 있다.

원본을 그냥 검색에 던지는 방식과 대비된다. 원본을 조각내 검색하면 조각이 문맥을 잃는다.



CC BY-SA 4.0 원본을 조각내 검색하는 방식 — 문서를 잘라 벡터로 만들고 질의와 가까운 조각을 찾아 넘긴다. 조각이 잘리면 문맥도 잘린다. 저작자 Gknor, 위키미디어 공용 `RAG_schema.svg` (2023-10-24), CC BY-SA 4.0.

Anthropic이 든 예시는 `The company's revenue grew by 3% over the previous quarter.` 한 줄만 남아 어느 회사, 어느 분기인지 알 수 없게 되는 경우다. 정제 노트는 제목·출처·요약을 갖고 있어 잘려도 문맥이 남는다. 그리고 노트가 늘어나도 `wiki/INDEX.md` 색인부터 읽으므로 긴 문서 중간에 정보가 묻히지 않는다.

인제스트의 보류 판단

인제스트는 옮기는 작업이 아니라 판단하는 작업이다. 들어온 것을 다 위키로 만들지 않는다.

2026-08-28 실행에서 `Clippings/` 2건을 처리했다. Reddit 게시글 1건에서는 wiki 노트 4건이 나왔고, `점검-결과-보고.md` (hwpix 변환 산출물, 403자)는 wiki 노트를 만들지 않았다. 실행 로그의 사유는 이렇다.

wiki 노트를 만들지 않았다. 실측이 아닌 합성 수치를 wiki article로 올리면 이 vault의 지식 계층에 검증 불가능한 도메인 사실이 들어간다. 원문은 `raw/점검-결과-보고.md` 로, 바이너리 원본은 `raw/hwp/점검-결과-보고.hwp` 로 보존해 변환 파이프라인의 증거로만 남긴다.

판정 기준은 하나다

다시 꺼내 쓸 개념인가. 그렇다면 노트로, 아니면 원본으로만 남긴다.

보류 사유는 같은 로그의 `Dropped / Issues` 에 남는다.

`점검-결과-보고.md` — wiki화 보류. 사유는 위 "처리 2". 실제 설비 점검 데이터가 들어오면 그때 도메인 노트를 세우는 편이 낫다.

사유를 적어 두기 때문에 나중에 "이 문서는 넣었는데 왜 위키에 없지"를 로그에서 찾을 수 있다.

이 섹션을 보면 쌓인 위키를 실제 업무에서 어떻게 꺼내 쓰는지 안다. 질의와 주간 보고서 두 가지를 시연하고, 마지막에 파일이 지나온 자리를 확인한다.

위키 질의

지금 이 vault의 위키 상태다.

노트	무엇
claude-code	구독 인증으로 도는 터미널 코딩 에이전트 CLI
claude-subscription-billing-boundary	구독·API 과금 경계와 누수 지점
hermes-agent	항상 켜져 있는 오케스트레이터 에이전트 런타임
multi-agent-role-separation	역할별로 도구를 나눠 맡기는 구성 패턴

아티클 4개는 1주차에 클리핑 1건을 넣은 결과다. 수강자가 **지난주에 한 일의 결과다. 남의 사례가 아니다.**

claude-subscription-billing-boundary.md 에는 status: needs-update 가 붙어 있다. 근거가 Reddit 게시물과 댓글뿐이고 1차 출처로 확인되지 않아서다. 위키는 완성품이 아니라 갱신 대상이다.

질의는 vault에서 ~ 정리해줘 로 시킨다.

과장하지 않는다

노트 4개로 답할 수 있는 범위는 좁다. vault-query 가 노트 몇 개부터 쓸모가 생기는데에 대한 실측 근거는 없다. 넣는 만큼 답이 좋아진다. 그게 과제의 이유다.

주간 보고서

트리거는 주간 보고 다. 산출은 areas/weekly/YYYY-MM-DD.md 와 같은 이름의 .html 뷰 두 장이다. 입력이 git 변경 이력이라 사람이 따로 적을 게 없다.

2026-08-28 실제 산출값이다.

항목	값
커밋 (전체 / 실작업)	20 / 14
머지된 PR	5
변경 파일	40
삽입 / 삭제	+3,876 / -115
신규 스킬 진입점	8
신규 wiki 문서	0

워크스트림은 4개로 묶였다 — 문서 변환 스킴 3종 반입, 온보딩 스크립트 실행기 결함 수정, 상위 vault 동기화 2회, README 반영. 각 워크스트림에 PR 번호와 날짜, 작업자가 붙는다.

숫자는 전수 집계다. 눈에 띄는 것만 고른 게 아니라 구간의 모든 커밋을 센다. 보고서 **Notes** 에 집계 명령과 경계 커밋을 함께 적어 두기 때문에 나중에 같은 값을 다시 뽑을 수 있다.

```
git diff --shortstat 584c45f..master
git log --no-merges 584c45f..master
```

신규 wiki 문서 0이라는 값이 특히 유용하다. 도구는 갖췄지만 지식 축적은 아직 시작되지 않았다는 뜻이고, 그게 지금 수강자의 위치이기도 하다.

결과 확인 — 파일 하나가 지나는 자리

오늘 시연한 파일 한 건이 지금 어디에 어떤 상태로 남았는지 여섯 자리를 확인한다.

위치	무엇이	이후
Clippings/	변환된 마크다운	인제스트가 원문을 raw/ 로 옮기면서 비워진다
raw/hwp/	원본 .hwp 그대로	수정·삭제하지 않는다
raw/	변환된 마크다운 원문	수정·삭제하지 않는다
wiki/	재사용할 개념만 노트로	사실이 바뀌면 갱신한다
wiki/INDEX.md	노트 목록 한 줄	질문할 때 여기부터 읽는다
outputs/runs/	실행 로그 — 판단과 보류 사유	왜 그랬는지의 근거

한 문장으로

원본은 raw/ 에 그대로, 쓸 만한 개념은 wiki/ 에 정리해서, 왜 그렇게 판단했는지는 로그에 남는다. 세 가지가 분리돼 있어서 잘못 정리해도 되돌릴 수 있다.

수강자는 이 자리에서 실습하지 않는다. 대신 자기 문서 1건으로 같은 경로를 직접 돌린다. 다음 주 시작할 때 확인한다.

자기 문서 1건

자기 업무 문서 1건을 변환부터 인제스트까지 통과시킨다.

- 01 **문서를 고른다.** .hwp가 가장 쉽다. 추가 설치가 없다. 회사 밖으로 나가면 안 되는 문서는 고르지 않는다
- 02 **변환하고 검수한다.** 게이트 3개를 통과시킨다. 첫 줄과 표를 눈으로 본다. 머리말이 본문에 섞였는지 확인한다
- 03 **인제스트한다.** 위키 노트가 생기는지, 아니면 보류되는지 본다. 보류됐으면 로그에 적힌 사유를 읽는다
- 04 **물어본다.** 새로 넣은 내용을 vault에 질문한다. 답이 노트를 근거로 나오는지 확인한다

채팅에 적을 문구는 이 세 줄이다.

이 한글 파일 마크다운으로 변환해서 Clippings에 넣어줘
인제스트해줘
vault에서 ~ 정리해줘

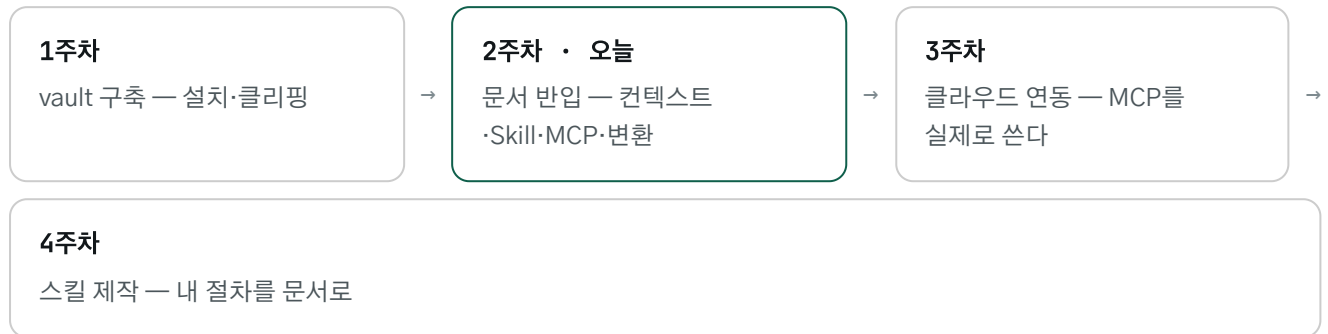
막히는 경우도 결과다

게이트 1에서 "아니오"가 맞는 문서였다면 그걸 알아챈 것도 성과다. 그 문서는 vault 밖 변환 모드로 변환만 하고 결과를 vault에 넣지 않는다.

다음 주 시작할 때 확인한다. 한 건이면 된다.

오늘 다룬 개념이 3주차와 4주차에서 어디로 이어지는지 위치를 잡는다.

3주차와 4주차



3주차는 클라우드 연동이다. Google 드라이브·슬라이드·시트·메일을 붙이고, 텔레그램·카카오톡으로 알림을 받는다. 오늘 본 MCP가 그 준비였다.

4주차는 스킬을 직접 만든다. 오늘 본 `SKILL.md` 형식으로 자기 업무 절차를 문서로 고정한다. 매번 설명하던 절차를 한번 적어 두고 그다음부터는 이름으로 부른다.

과제는 문서 1건이다. 다음 주 시작할 때 확인한다.

이 자료의 근거와 한계

2주차 교육 자료 · 2026-08-28 작성. 수치는 작성 시점 실측이며 근거 문서는 `projects/second-brain/outputs/week2-sources/`에 있다. 본문의 § 표기는 그 시트의 절을 가리킨다.

발표 전 확인할 것

- `vault-query`를 노트 4개로 미리 한 번 돌려 본다. 답이 빈약하면 그 사실을 그대로 보여주는 쪽이 낫다
- 게이트 문구가 실제 화면에 스킬 문서 원문대로 뜨는지 리허설로 확인한다
- 주간 보고서 `.html` 뷰를 브라우저로 한 번 열어 본다
- `chars ≥ 300` 임계는 `needs-update` 상태다. 이 사실까지 말할지는 시간에 따라 판단한다
- 머리말 중복은 이 vault의 실측 1건이 유일한 근거다. "확률이 높다"까지만 말하고 단정하지 않는다
- Obsidian에서 `Clippings/` 파일 탭을 닫아 둔다. 열려 있으면 빈 파일이 다시 생겨 인제스트 건수가 어긋난다

그림 출처 — MCP 구조도와 도구 선적재 도해는 `modelcontextprotocol.io`, CC BY 4.0(저작자 표시). RAG 구조도는 위키미디어 공용, 저작자 Gknor, CC BY-SA 4.0(저작자 표시·동일 라이선스). 컨텍스트 로드 시점·`SKILL.md` 구조·3단 로딩 표는 Anthropic 공식 문서와 엔지니어링 블로그의 도해로, 이용 허락 표기가 없어 **사내 비공개 교육 인용 범위로만 쓴다**. 외부 공개 시 재판단이 필요하다.